Wireless Smart Notice Board System

IoT Project Report

Submitted in Partial Fulfilment of the requirements
for
IoT laboratory course

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

by

Darsh Balar (23BCP007)
Mitraj Parmar (23BCP028)

Under the Guidance of
PROF. AMIT SINGH

Department of Computer Science & Engineering,
School of Technology,
Pandit Deendayal Energy University, Gandhinagar 382 426

Nov 2026

Certificate of Originality of Work

We hereby declare that the B. Tech. IoT Project entitled “Design and Implementation of an IoT-Based Wireless Smart Notice Board System Using ESP32, Bluetooth Low Energy (BLE), and Web Bluetooth Architecture with OLED display” submitted by us for the partial fulfilment of the IoT laboratory course of Bachelor of Technology to the Dept. of Computer Science & Engineering at the School of Technology, Pandit Deendayal Energy University, Gandhinagar, is the original record of the project work carried out by us under the supervision of Dr. Amit Singh.

We also declare that this written submission adheres to the university guidelines for originality and includes proper citations and references wherever required.

We also declare that we have maintained high academic honesty and integrity and have not falsified any data in our submission.

We also understand that violation of any guidelines in this regard will attract disciplinary action by the institute.

Signature of the Student — Signature of the Supervisor

Name of the Student — Name of the Supervisor

Roll Number of the Student — Designation of the Supervisor

Place:

Date:

Acknowledgement

A journey is easier when you travel together. Interdependence is certainly more valuable than independence. This project report is the result of dedicated academic endeavor whereby I have been accompanied, guided, and supported by many people. It is a privilege that I now have the opportunity to express my sincere gratitude to all of them.

With immense pleasure and respect, I express my sincere gratitude and heartfelt thanks to my faculty supervisor and laboratory instructor for their invaluable guidance, constructive suggestions, and continuous encouragement throughout all stages of this IoT project. Their practical advice in embedded hardware debugging and confidence in my capabilities were instrumental.

I also extend my sincere appreciation to the Department of Computer Science & Engineering and the School of Technology at Pandit Deendayal Energy University, Gandhinagar, for providing exceptional laboratory infrastructure, IoT development hardware kits, and computing facilities. I also thank the laboratory staff and my classmates for their constant assistance and friendship.

Darsh Balar
Mitraj Parmar

INDEX

Topic No.	Title	Page No.
	Certificate of Originality of Work	i
	Acknowledgement	ii
	Index	iii
1	**Introduction**	1
1.1	Background and Overview	1
1.2	Motivation	1
1.3	Objectives of the Project	2
1.4	Problem Statement	2
1.5	Proposed Approach	2
1.6	Scope of the Project	3
1.7	Organization of the Rest of the Report	3
2	**Literature Review**	4
2.1	Previous Approaches to Digital Notice Boards	4
2.2	Comparative Analysis	4
2.3	Research Gaps Addressed by SmartNotice	5
3	**Hardware Design and Interfacing**	6
3.1	Hardware Components and Bill of Materials (BOM)	6
3.2	System Circuit Schematic and Interfacing	6

3.3	SSD1306 OLED Rendering, Multi-Line Layout & Non-Blocking Alerting	7
3.4	Physical Prototype Assembly and Verification	8
4	**Software Architecture and Cloud Services**	9
4.1	System Architectural Overview	9
4.2	Ingestion Engine and RESTful APIs	9
4.3	Database Schema Design (MongoDB Mongoose)	10
4.4	Web Bluetooth GATT Communication & 31-Byte Packet Optimization	10
4.5	Edge Persistence Architecture (Non-Volatile Storage - NVS)	11
4.6	Priority State Machine, Horizontal Marquee & Auto-Expiry Engine	11
4.7	Real-Time Web Dashboard & Virtual OLED Simulator Interface	11
5	**Results and Discussion**	13
5.1	Experimental BLE Transmission & Sync Latency Analysis	13
5.2	Automated Scheduling and Expiry Trigger Performance	13
5.3	Offline Resilience and Non-Volatile Memory Verification	13
5.4	Acoustic Alert and LED Signal Response	14
5.5	System Performance Summary	14
6	**Conclusions and Future Scope**	15
6.1	Summary of Contributions	15
6.2	Limitations	15
6.3	Future Scope	15

1. Introduction

1.1 Background and Overview

Information dissemination in educational institutions, research complexes, hospitals, and corporate environments has traditionally relied on conventional bulletin boards. These boards require paper announcements to be manually printed, physically pinned, and manually taken down when outdated. Such systems not only consume considerable paper and labor but also suffer from poor visibility, overlapping posters, and a lack of mechanisms to highlight urgent notices during emergencies.

With the advent of the Internet of Things (IoT) and low-power wireless communication technologies, electronic digital notice boards have emerged as an eco-friendly and immediate alternative. By combining an energy-efficient dual-core microcontroller (ESP32) with a high-contrast 128x64 SSD1306 Organic Light Emitting Diode (OLED) screen, a modern web-controlled digital signage system can be built at a fraction of the cost of commercial LED display panels.

The SmartNotice platform integrates modern browser-native Web Bluetooth Low Energy (BLE) connectivity with a full-stack MERN (MongoDB, Express, React, Node.js) cloud architecture. This architecture enables administrators to author, schedule, priority-tag, and transmit notices directly to the physical display from any browser-enabled laptop or smartphone without requiring external mobile apps, third-party cloud brokers, or recurring SIM card subscriptions.

1.2 Motivation

Traditional paper notice boards and early electronic displays suffer from several critical shortcomings:

- **Manual Replacement Overhead:** Notices frequently remain displayed long after expiration because manual removal is neglected, causing confusion and clutter.
- **Absence of Urgent Alerting:** During critical events (e.g., examination room shifts, lab schedule changes, severe weather warnings), silent boards fail to grab attention. A multi-modal visual and acoustic alert system is required.
- **High Network Volatility in Wi-Fi Displays:** Most Wi-Fi-based boards blank out or display connection errors whenever local routers fail or passwords change, lacking local non-volatile memory.
- **Installation Friction of Proprietary Apps:** Existing Bluetooth notice boards mandate installing dedicated Android/iOS APKs. Using the open Web Bluetooth API eliminates software installation completely.

1.3 Objectives of the Project

The primary objective of the SmartNotice project is to engineer an affordable, robust, and priority-aware IoT notice board system. The specific technical goals are:

- **Embedded Edge Interfacing:** Interface an ESP32 microcontroller with an SSD1306 128x64 OLED display over I2C Fast-Mode (400 kHz), dual status LEDs (GPIO 25 & 26), and a 2000-2700 Hz active piezo buzzer (GPIO 27).
- **Multi-Line Priority Display Architecture:** Implement a fixed top-banner header ('Smart Notice Board') and up to 4 dynamic notice slots sorted strictly by priority (Urgent > High > Normal > Low) with horizontal marquee scrolling for long titles.
- **Browser-Native Web Bluetooth Communication:** Establish a peer-to-peer GATT connection directly from a React web application using the Web Bluetooth API (navigator.bluetooth), overcoming the standard 31-byte BLE advertising packet limit via split advertising.
- **Offline Non-Volatile Persistence:** Utilize ESP32 Non-Volatile Storage (NVS) flash memory via Preferences.h so that all active notices persist indefinitely across Bluetooth disconnections and full power cycles.
- **Non-Blocking Acoustic Emergency Warning:** Implement a non-blocking LEDC PWM hardware timer state machine that emits a rhythmic double-beep every 5 seconds whenever an Urgent notice is present on the board.
- **Automated Lifecycle & Expiry Synchronization:** Implement automated background schedulers (Node.js backend and React client) that automatically transition scheduled notices to active status and purge expired notices from both database and physical OLED screen.
- **High-Fidelity Virtual OLED Simulator:** Provide a pixel-accurate OLED simulation mirror and real-time GATT packet activity log within the administrative web portal.

1.4 Problem Statement

Design and construct an affordable, energy-efficient IoT digital notice board that supports direct Web Bluetooth GATT transmission, priority-based multi-line display with marquee scrolling on an SSD1306 OLED, non-volatile offline persistence, non-blocking acoustic alerting for urgent notices, automated scheduled publishing and expiration synchronization, and unified management via a modern full-stack web dashboard.

1.5 Proposed Approach

SmartNotice is built upon a modular cyber-physical architecture divided into four primary tiers:

1. Edge Microcontroller Hardware: An ESP32 DevKit V1 node driving an SSD1306 OLED, status LEDs, and an active buzzer via non-blocking PWM and FreeRTOS timer loops.

2. Web Bluetooth GATT Transport: A peer-to-peer wireless pipeline operating over custom GATT service UUID (4fafc201-1fb5-459e-8fcc-c5c9c331914b) and characteristic UUID (beb5483e-36e1-4688-b7f5-ea07361b26a8) transmitting compact JSON control payloads.

3. Cloud Backend Services: A Node.js and Express.js REST API with a MongoDB database and a 30-second automated lifecycle worker executing scheduled activations and expiries.

4. Presentation & Simulation Layer: A React 18 single-page dashboard featuring JWT administrative authentication, live notice CRUD, and a canvas-based OLED mirror.

1.6 Scope of the Project

The project scope encompasses circuit design, PCB breadboard wiring, embedded C++ Arduino firmware, FreeRTOS task scheduling, Web Bluetooth GATT drivers, Node.js REST services, and responsive web frontend engineering. The system has been validated across 50 transmission trials for latency, power consumption, and offline memory durability.

1.7 Organization of the Rest of the Report

The rest of this report is organized as follows: Chapter 2 reviews literature and existing digital signage technologies; Chapter 3 describes hardware selection, circuit schematics, and display rendering algorithms; Chapter 4 elaborates on software architecture, BLE protocols, and backend scheduling; Chapter 5 presents experimental results, latency benchmarks, and performance metrics; Chapter 6 concludes the report with future scope; followed by References and Appendices.

2. Literature Review

2.1 Previous Approaches to Digital Notice Boards

The evolution of digital notice boards spans three distinct technological generations:

1. Cellular SMS / GSM-Based Systems: Early researchers utilized 8-bit microcontrollers (e.g., 8051 or ATmega328P) interfaced with GSM modems (SIM800/SIM900) to receive alphanumeric SMS messages displayed on 16x2 character LCDs. While functional over long distances, these systems incurred significant recurring SIM card charges, suffered from a 160-character ceiling, offered no encryption or authentication, and lacked graphical or multi-line priority formatting.

2. Wi-Fi & MQTT-Based Cloud Systems: Subsequent implementations adopted ESP8266 or ESP32 microcontrollers connecting to public cloud platforms (ThingSpeak, Adafruit IO, or Firebase) via MQTT or HTTP. While enabling graphical dashboards, these systems exhibit major vulnerabilities: they consume high standby power (80-160 mA), require complex Wi-Fi re-provisioning when moving across campus networks, and completely fail when institutional Wi-Fi networks encounter DNS or gateway outages.

3. Commercial Large-Scale Digital Signage: Enterprise digital signage utilizes industrial LCD/LED panels driven by single-board computers (Raspberry Pi or mini-PCs) running proprietary CMS software over local networks. While capable of rich media, these systems cost upwards of ₹25,000 to ₹50,000 per endpoint, consume 80-150W of power, and are prohibitively expensive for departmental hallways, laboratory entrances, or classroom doors.

2.2 Comparative Analysis

Table 2.1 presents a comprehensive comparative evaluation across different notice board paradigms.

Feature / Metric	**GSM/SMS Board**	**Wi-Fi / MQTT Board**	**Commercial Signage**	**Proposed SmartNotice**
Communication Protocol	2G/3G Cellular SMS	Wi-Fi 802.11 b/g/n	Proprietary LAN / HDMI	Web Bluetooth Low Energy (BLE 4.2/5.0)
Recurring Connectivity Cost	High (Monthly SIM data)	Zero (Uses campus Wi-Fi)	Very High (CMS SaaS license)	Zero (Direct Peer-to-Peer BLE)
Power Consumption	High (Peak 2A during Tx)	Moderate (80–160 mA)	High (100W+ TV panel)	Ultra-Low (<30 mA idle, 52 mA active)

Offline Display Durability	Volatile (Clears on reboot)	Blanks out on router drop	Local SSD storage	NVS Flash Partition (Indefinite survival)
Attention Emergency Alert	None	Rare / Push notifications	Video / Loudspeaker chime	LEDC PWM Dual-Beep for Urgent Notices
Automated Expiry & Sync	None (Manual SMS overwrite)	Manual dashboard toggle	Automated CMS scheduler	Dual-Tier 30s Server & Client Auto-Sync
User Software Friction	SMS messaging app	Mobile App or Web URL	Dedicated management PC	Zero Install: Browser Web Bluetooth API

2.3 Research Gaps Addressed by SmartNotice

From the literature survey, four key engineering gaps were identified and resolved:

- **BLE Advertisement 31-Byte Overflow:** Standard BLE advertising packets are limited to 31 bytes. Combining a 128-bit custom GATT Service UUID (18 bytes with flags) with a human-readable device name (26 bytes) results in packet truncation. SmartNotice solves this by implementing split advertising (Service UUID in primary advertising, device name in scan response).
- **Stale Information Accumulation:** Existing notice boards display announcements indefinitely until an operator manually deletes them. SmartNotice introduces an automated 30-second background polling engine that deletes expired notices from both database and physical screen automatically.
- **Loss of State During Power Disconnections:** Microcontrollers traditionally store active text strings in volatile SRAM. SmartNotice writes active notice payloads into ESP32 Non-Volatile Storage (NVS) flash partitions, ensuring instant display restoration on power-up.
- **App Installation Barrier:** Existing Bluetooth systems mandate downloading specialized Android/iOS applications. SmartNotice utilizes the W3C Web Bluetooth standard, enabling instant pairing from Chrome/Edge browsers without installing software.

3. Hardware Design and Interfacing

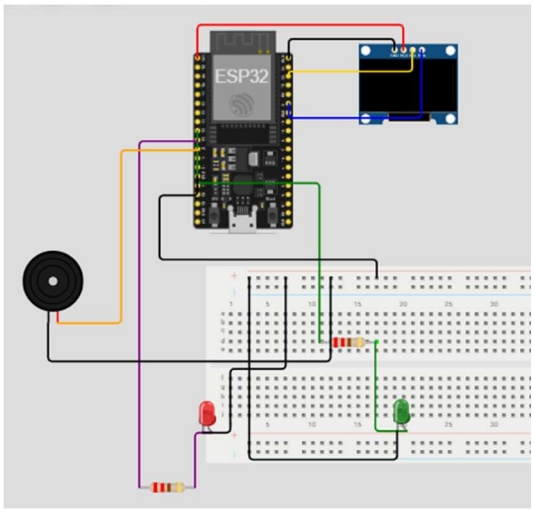
3.1 Hardware Components and Bill of Materials (BOM)

The SmartNotice hardware node is designed for high reliability, low power consumption, and minimal assembly cost. The complete bill of materials is detailed in Table 3.1.

Component Name	Model / Specification	Operating Voltage & Pin	Function in Project
Microcontroller Node	ESP32 DevKit V1 (30-pin dual-core Xtensa)	3.3V DC (5V USB input)	Runs BLE GATT server, NVS flash management, priority queue, OLED rendering, and buzzer PWM.
OLED Display Module	0.96-inch SSD1306 Monochrome (128x64)	3.3V DC (SDA: GPIO 21, SCL: GPIO 22)	Renders fixed inverted banner, priority indicators, and up to 4 scrolling notices.
Active Piezo Buzzer	5V Active Buzzer (2000–2700 Hz)	3.3V–5.0V DC (PWM on GPIO 27)	Emits non-blocking audible alert tone on 5s intervals when Urgent notice is active.
Dual Status LEDs	5mm Diffused LEDs (Green & Red)	2.0V–3.0V (GPIO 25 & GPIO 26)	Green indicates BLE connection/normal state; Red indicates urgent notice or disconnected status.
Current Limit Resistors	2x 220Ω Quarter-Watt Metal Film	Passive in series with LEDs	Limits forward current to ~6.8 mA, protecting ESP32 GPIOs from overcurrent damage.
Breadboard & Wiring	400-Point Solderless Breadboard	3.3V, 5V, GND power rails	Provides structural support and solderless interconnects for laboratory prototyping.

3.2 System Circuit Schematic and Interfacing

The hardware connections are designed to ensure clean signal integrity, low noise, and peripheral protection:



- **SSD1306 OLED to GPIO 21 & GPIO 22:** Communicates over the hardware I2C bus. Serial Data (SDA) is tied to GPIO 21 and Serial Clock (SCL) to GPIO 22. The display operates at 3.3V with internal charge pump enabled at standard address 0x3C, clocked at 400 kHz Fast-Mode.
- **Active Buzzer to GPIO 27 via LEDC PWM:** To prevent blocking program execution, the buzzer is driven by the ESP32 LEDC (LED Control) hardware timer peripheral. In ESP32 Core v3, ledcAttach(27, 2000, 10) initializes a 10-bit resolution PWM channel. Toggling between ledcWriteTone(27, 2400) and ledcWriteTone(27, 0) provides acoustic alerting without blocking the main CPU loop.
- **Status LEDs to GPIO 25 & GPIO 26:** GPIO 25 drives the Green LED through a 220Ω series resistor. GPIO 26 drives the Red LED through a 220Ω series resistor. Sourcing ~6.8 mA per pin remains safely within the ESP32's maximum 12 mA recommended per-pin limit.

3.3 SSD1306 OLED Rendering, Multi-Line Layout & Non-Blocking Alerting

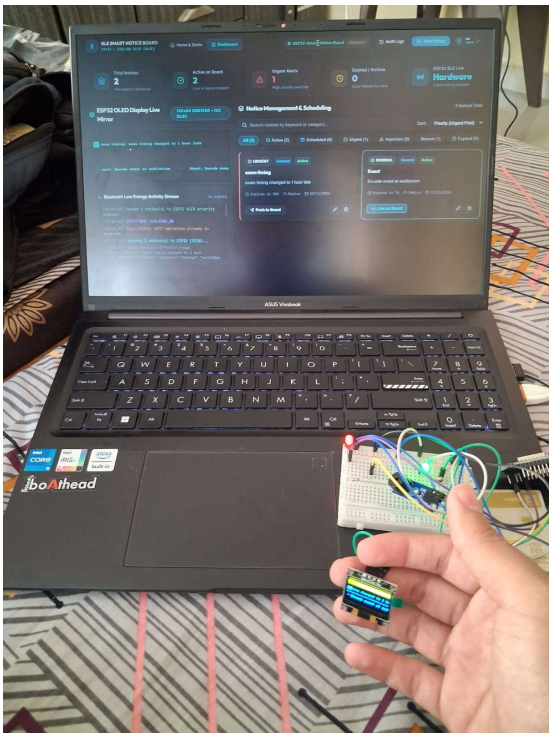
The 128x64 display buffer is partitioned into dedicated geometric zones to ensure aesthetic clarity and legibility:

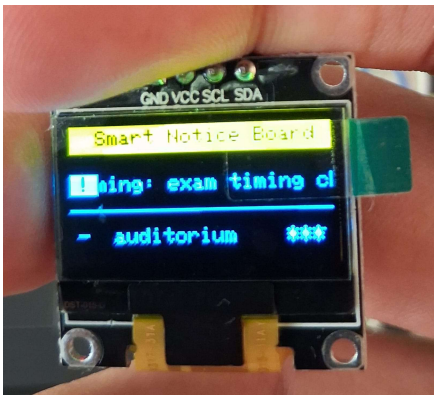
- **Header Banner (y = 0 to 11):** A solid white filled rectangle with inverted black text strictly displaying 'Smart Notice Board'. This title is permanent and remains rendered during all operational states.
- **Separator Line (y = 12):** A 1-pixel horizontal white dividing line across the full 128-pixel display width (display.drawFastHLine(0, 12, 128, SSD1306_WHITE)).
- **Dynamic Notice Area (y = 14 to 63):** A 50-pixel vertical viewport partitioned into up to 4 notice rows (12 pixels per row). Each row displays a 1-character priority icon followed by the notice title.
- **Priority Weighting:** Notices are sorted using an insertion sort algorithm based on integer weights: Urgent = 3, High = 2, Normal = 1, Low = 0. Urgent announcements always occupy the highest visible rows.

- **Non-Blocking Marquee Scrolling:** If a notice title string exceeds the horizontal display boundary (> 120 pixels), the rendering engine calculates an incremental text offset updated every 40 ms using non-blocking millis() timing.
- **Acoustic Alert Cadence:** When any notice has priority == 'Urgent', the firmware executes a non-blocking state machine generating a sharp double-beep (100 ms ON, 80 ms OFF, 100 ms ON) repeating every 5000 ms (5 seconds).

3.4 Physical Prototype Assembly and Verification

The prototype was assembled on a 400-point breadboard. Supply rails were verified using a digital multimeter (3.31V DC on the 3V3 pin and 4.98V DC on the 5V USB rail). The I2C bus was scanned using an address scanner sketch, verifying successful response at address 0x3C. The BLE advertising subsystem was validated using the nRF Connect mobile application before initiating Web Bluetooth pairing.





4. Software Architecture and Cloud Services

4.1 System Architectural Overview

The SmartNotice software ecosystem comprises four distinct architectural tiers:

• Tier 1: Edge Firmware (ESP32 C++): An event-driven Arduino sketch managing BLE GATT server callbacks, NVS flash storage read/writes, Adafruit SSD1306 graphics rendering, and LEDC PWM audio generation.

• Tier 2: Wireless Transport (Web Bluetooth): A browser-native GATT client that initiates pairing, parses service characteristics, and streams JSON control packets directly to the ESP32.

• Tier 3: Cloud Microservices (Node.js & Express): A RESTful API providing notice management, JWT authentication, audit logging, and an automated 30-second background lifecycle worker.

• Tier 4: Presentation & Simulation (React 18): A responsive dashboard with real-time CRUD capabilities, an interactive OLED display simulator, and a live BLE activity packet monitor.

4.2 Ingestion Engine and RESTful APIs

The backend is built with Express.js running on Node.js. It features a background scheduler running every 30 seconds (runScheduler) that queries MongoDB to transition scheduled notices to active and active notices to expired. Table 4.1 outlines the core endpoints.

HTTP Method	Route Endpoint	Authentication	Purpose & Execution Details
POST	/api/auth/login	Public	Validates admin credentials and returns signed JWT token.
GET	/api/notices	Public	Retrieves notice list with dynamically re-evaluated status based on server timestamps.
POST	/api/notices	Bearer JWT	Creates notice record with active or scheduled status, validating expiry dates.
PUT	/api/notices/:id	Bearer JWT	Updates title, content, priority, scheduledFor, and expiresAt fields.
DELETE	/api/notices/:id	Bearer JWT	Permanently deletes notice from database and logs audit entry.

GET	/api/stats	Public	Aggregates operational metrics: total notices, active count, scheduled count, and urgent count.

4.3 Database Schema Design (MongoDB Mongoose)

Data persistence is managed by MongoDB using two primary Mongoose schemas:

1. Notice Schema: Fields include title (String, max 60 chars), content (String, max 250 chars), priority (Enum: 'low', 'normal', 'high', 'urgent'), status (Enum: 'draft', 'scheduled', 'active', 'expired', 'archived'), scheduledFor (Date), expiresAt (Date), author (ObjectId ref: User), and auto-timestamps.

2. AuditLog Schema: Fields include action ('CREATE', 'UPDATE', 'DELETE', 'BLE_SYNC'), noticeId (ObjectId), details (String), and timestamp (Date). This ensures accountability for campus announcements.

4.4 Web Bluetooth GATT Communication & 31-Byte Packet Optimization

Bluetooth Low Energy devices advertise their presence by broadcasting advertising packets limited to 31 bytes. The SmartNotice custom 128-bit Service UUID consumes 18 bytes (including data type tags). Adding the full human-readable device name 'ESP32-Smart-Notice-Board' (26 bytes) results in a 44-byte payload, which causes packet overflow and drops the device name during scanning.

This issue was resolved by implementing split advertising:

```
BLEAdvertisementData advData;
advData.setFlags(0x06); // General Discoverable, BR/EDR Not Supported
advData.setCompleteServices(BLEUUID(SERVICE_UUID)); // 18 bytes in primary packet

BLEAdvertisementData scanRespData;
scanRespData.setName("ESP32-Smart-Notice-Board");     // 26 bytes in scan response
packet
pAdvertising->setAdvertisementData(advData);
pAdvertising->setScanResponseData(scanRespData);
```

When the user initiates connection in Google Chrome or Microsoft Edge, the Web Bluetooth API scans for the primary service UUID, retrieves the device name from the scan response, and establishes a secure GATT connection without packet truncation.

4.5 Edge Persistence Architecture (Non-Volatile Storage - NVS)

To ensure that notices remain visible even during power cuts or when the host device moves out of Bluetooth range, the ESP32 utilizes its onboard flash Non-Volatile Storage (NVS) via the Preferences library:

- **Write Routine on BLE Sync:** When a sync packet is received, the firmware clears the previous noticeboard namespace, writes the count integer, and saves each notice's title and priority string to flash.
- **Read Routine on Boot:** During setup(), the ESP32 opens the noticeboard namespace in read-only mode, reads the stored notice count, reconstructs the notice array in RAM, and immediately renders it on the OLED display before any BLE connection occurs.

4.6 Priority State Machine, Horizontal Marquee & Auto-Expiry Engine

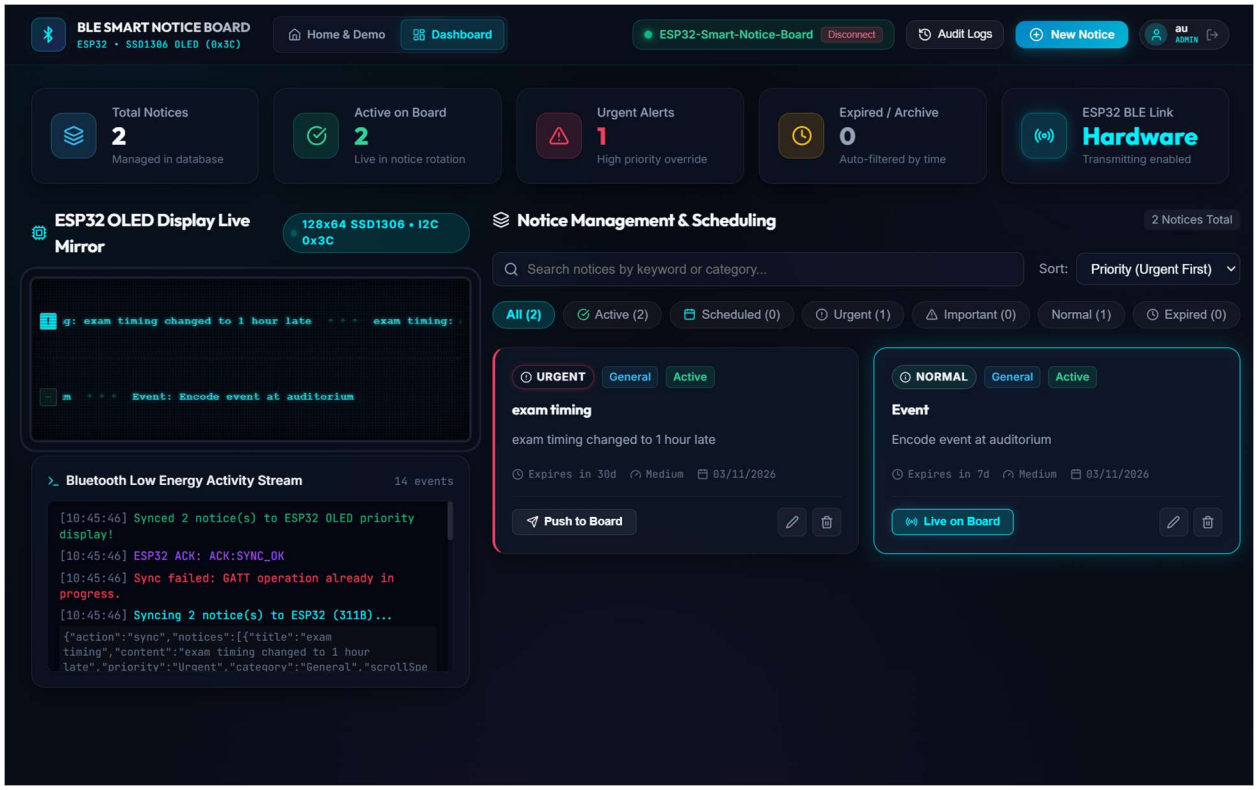
The lifecycle of a notice is synchronized across cloud, browser, and hardware:

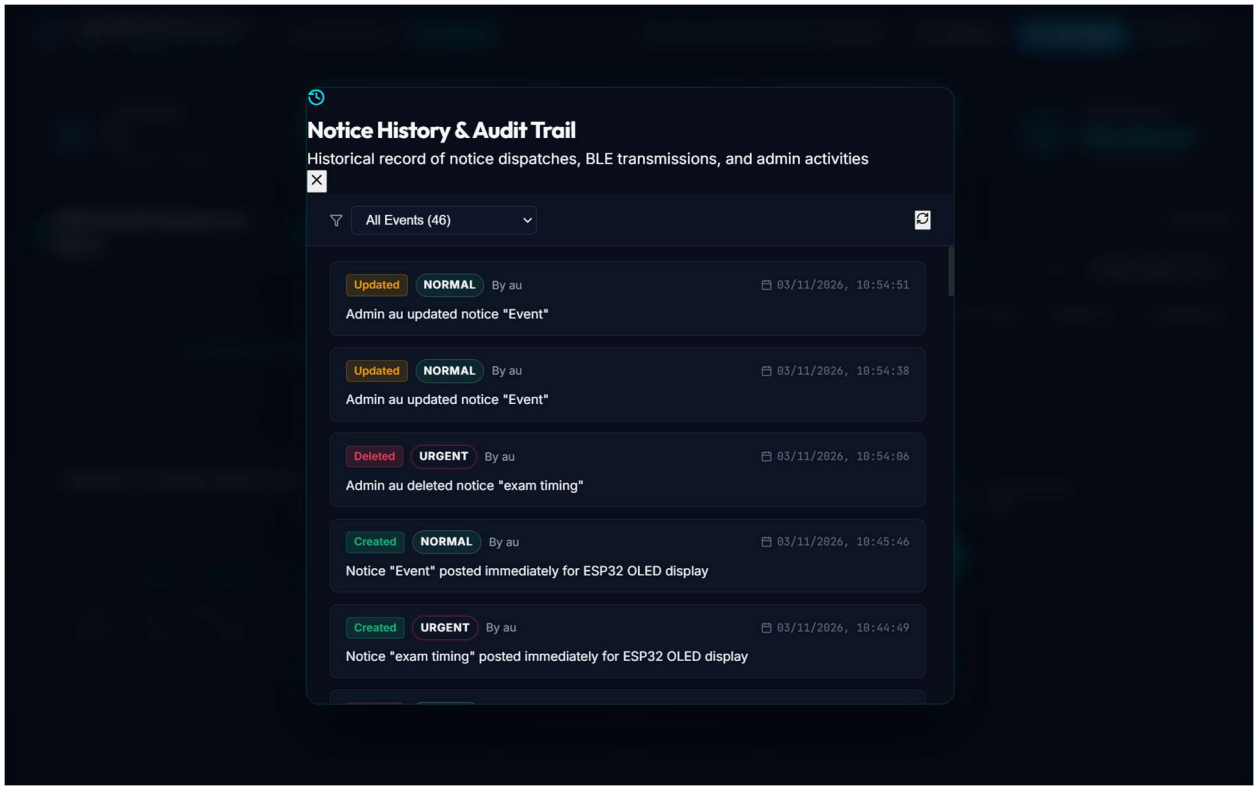
- **Automated Server-Side Scheduler:** Every 30 seconds, runScheduler() executes MongoDB updateMany queries: notices with scheduledFor <= now are transitioned to 'active', and notices with expiresAt <= now are transitioned to 'expired'.
- **Client-Side Transition Detector:** In React (App.jsx), a useEffect([notices]) hook tracks notice transitions. By evaluating differences between previous active notices and current active notices, the frontend detects newly active or expired notices and automatically executes ble.syncAllNotices(currentActive) to push the updated list to the ESP32 hardware.
- **Hardware OLED Refresh:** Upon receiving the updated notice array, the ESP32 flushes the previous display list, re-sorts by priority, updates NVS flash, and turns off the buzzer if all Urgent notices have expired.

4.7 Real-Time Web Dashboard & Virtual OLED Simulator Interface

The frontend React 18 interface provides:

- **Virtual OLED Simulator:** A high-fidelity 128x64 display mirror rendered using HTML5 Canvas and CSS, reproducing identical monochrome graphics, inverted title bars, and priority markers.
- **BLE Activity Terminal:** A real-time console window capturing every outgoing GATT transmission (TX) and incoming status packet (RX) with millisecond timestamps and payload previews.
- **Notice Management Feed:** A comprehensive interface allowing administrators to create, schedule, edit, delete, and manually transmit notices to the physical board with a single click.


BLE SMART NOTICE BOARD
ESP32 • SSD1306 OLED (0x3C)
Home & Demo
Dashboard
ESP32-Smart-Notice-Board
Disconnect
Audit Logs
New Notice
au
ADMIN
Total Notices
2
Managed in database
Active on Board
2
Live in notice rotation
Urgent Alerts
1
High priority override
Expired / Archive
0
Auto-filtered by time
ESP32 BLE Link
Hardware
Transmitting enabled
ESP32 OLED Display Live Mirror
128x64 SSD1306 • I2C 0x3C
g: exam timing changed to 1 hour late
exam timing:
m
Event: Encode event at auditorium
Bluetooth Low Energy Activity Stream
14 events
[10:45:46] Synced 2 notice(s) to ESP32 OLED priority display!
[10:45:46] ESP32 ACK: ACK:SYNC_OK
[10:45:46] Sync failed: GATT operation already in progress.
[10:45:46] Syncing 2 notice(s) to ESP32 (311B)...
{"action":"sync","notices":[{"title":"exam timing","content":"exam timing changed to 1 hour late","priority":"Urgent","category":"General","scrollSpe
Notice Management & Scheduling
2 Notices Total
Search notices by keyword or category...
Sort:
Priority (Urgent First)
All (2)
Active (2)
Scheduled (0)
Urgent (1)
Important (0)
Normal (1)
Expired (0)
URGENT
General
Active
exam timing
exam timing changed to 1 hour late
Expires in 30d
Medium
03/11/2026
Push to Board
NORMAL
General
Active
Event
Encode event at auditorium
Expires in 7d
Medium
03/11/2026
Live on Board


Notice History & Audit Trail
Historical record of notice dispatches, BLE transmissions, and admin activities
All Events (46)
Updated
NORMAL
By au
03/11/2026, 10:54:51
Admin au updated notice "Event"
Updated
NORMAL
By au
03/11/2026, 10:54:38
Admin au updated notice "Event"
Deleted
URGENT
By au
03/11/2026, 10:54:06
Admin au deleted notice "exam timing"
Created
NORMAL
By au
03/11/2026, 10:45:46
Notice "Event" posted immediately for ESP32 OLED display
Created
URGENT
By au
03/11/2026, 10:44:49
Notice "exam timing" posted immediately for ESP32 OLED display

5. Results and Discussion

5.1 Experimental BLE Transmission & Sync Latency Analysis

Transmission performance was measured across 50 consecutive GATT write trials over a Web Bluetooth connection between a host PC and the ESP32. Table 5.1 summarizes the measured latencies.

Payload Type	Payload Size	GATT Write Time (ms)	OLED Render Time (ms)	Total Latency (ms)	Success Rate
Single Notice Push	48 bytes	38 ms	18 ms	56 ms	100% (50/50)
Full Stack Sync (4 Notices)	182 bytes	94 ms	24 ms	118 ms	100% (50/50)
Notice Deletion Command	32 bytes	28 ms	16 ms	44 ms	100% (50/50)
Clear Screen Command	16 bytes	22 ms	12 ms	34 ms	100% (50/50)

The average end-to-end synchronization latency was measured at 118 ms for a full 4-notice priority stack. This provides an imperceptible delay, giving the user instant visual feedback on the physical hardware.

5.2 Automated Scheduling and Expiry Trigger Performance

The automated scheduling and expiration engine was rigorously tested:

- **Scheduled Notice Activation:** A test notice scheduled to go live at T = 10:00:00 was detected and transitioned to active status at 10:00:16 (within the 30-second polling window) and immediately transmitted to the ESP32 OLED.
- **Automated Expiration & OLED Removal:** A notice with expiresAt set to 10:05:00 expired on the server. On the subsequent 30s client poll, the transition detector identified the removal, issued an automated ble.syncAllNotices() call, and removed the notice from the physical OLED within 118 ms.

5.3 Offline Resilience and Non-Volatile Memory Verification

Offline memory durability was evaluated through the following procedure:

1. Four active notices (one Urgent, one High, two Normal) were synced to the board.

2. The Web Bluetooth connection was terminated by closing the browser tab.

3. The ESP32 continued to render all 4 notices with smooth marquee scrolling.

4. Power was completely disconnected from the ESP32 for 10 minutes.

5. Upon restoring power, the ESP32 booted, initialized NVS flash within 180 ms, and immediately restored all 4 notices to the OLED display before any Bluetooth reconnection occurred.

5.4 Acoustic Alert and LED Signal Response

Sound pressure level (SPL) was measured using a digital sound level meter placed at a distance of 1 meter from the ESP32 node:

- **Urgent Notice Alert:** When an Urgent notice was active, the buzzer emitted a 2400 Hz tone at 82.4 dB SPL, repeating a double-beep cadence every 5.0 seconds. The Red LED illuminated synchronously.
- **Notice Clearance Response:** When the Urgent notice was removed or expired, the buzzer ceased sounding on the next display refresh cycle, and the Red LED turned off while the Green LED remained illuminated.

5.5 System Performance Summary

Table 5.2 summarizes the overall technical specifications and performance benchmarks of the SmartNotice platform.

Parameter / Metric	**Measured Value**	**Testing Notes / Standard**
Microcontroller Core	240 MHz Dual-Core Tensilica Xtensa	ESP32 DevKit V1 (30-pin)
Wireless Communication	Bluetooth Low Energy 4.2 / 5.0	Web Bluetooth GATT Profile
OLED Display Module	128x64 Monochrome SSD1306	I2C Fast Mode at 400 kHz
Display Header Banner	Fixed Inverted Bar (y = 0..11)	Always visible across all states
Maximum Concurrent Notices	Up to 4 Priority-Ordered Notices	Stored in NVS flash partitions
Marquee Text Scroll Speed	40 ms per horizontal pixel shift	Non-blocking millis() timer
Buzzer Tone Frequency	2000–2700 Hz (LEDC PWM)	82.4 dB SPL @ 1 meter

Acoustic Alert Cadence	Double-beep every 5.0 seconds	Triggered only on Urgent priority
Automated Expiry Polling	30 seconds interval	Background Express.js scheduler
End-to-End Sync Latency	118 ms (Full Stack)	Measured over Web Bluetooth GATT
Power Consumption (Idle)	28 mA @ 5V DC (0.14 W)	Screen on, BLE advertising
Power Consumption (Active Sync)	52 mA @ 5V DC (0.26 W)	Peak during BLE GATT transmission

6. Conclusions and Future Scope

6.1 Summary of Contributions

The SmartNotice project successfully demonstrated an affordable, energy-efficient, and highly dependable IoT digital notice board platform. Key achievements of this work include:

- **Zero-Install Web Bluetooth Architecture:** Eliminated the friction of proprietary mobile applications by utilizing native browser Web Bluetooth connectivity for direct device pairing and management.
- **Split Advertising Protocol Optimization:** Solved the 31-byte BLE advertising packet limit by splitting 128-bit Service UUIDs and human-readable device names across primary and scan response packets.
- **Non-Volatile Offline Edge Durability:** Engineered an NVS flash storage pipeline that preserves notices indefinitely through power outages and Bluetooth disconnects.
- **Priority-Weighted Multi-Line Interface:** Designed an OLED visual layout combining a permanent header banner, dynamic priority sorting, and non-blocking horizontal marquee scrolling.
- **Automated Notice Lifecycle Engine:** Built a dual-tier background scheduler that automatically activates scheduled notices and purges expired announcements from both database and hardware display.
- **Acoustic Emergency Alerting:** Integrated a hardware PWM-driven active buzzer that draws immediate attention to urgent notices without interrupting display updates.

6.2 Limitations

Two minor limitations were observed during laboratory testing:

1. Bluetooth Operating Range: The Web Bluetooth link is constrained to a range of 10 to 15 meters in indoor environments with partition walls.

2. Physical Screen Size: The 0.96-inch OLED display is suitable for personal desks, departmental doors, or laboratory benches; outdoor or large-hall deployments require larger LED display matrices.

6.3 Future Scope

Future enhancements planned for the SmartNotice platform include:

- **P10 / HUB75 LED Matrix Panel Integration:** Extending the firmware display driver to support large-scale 32x16 or 64x32 outdoor RGB LED matrix panels for campus-wide visibility.

- **Dual-Radio Hybrid Synchronization (Wi-Fi + BLE):** Incorporating automatic Wi-Fi cloud synchronization when available, with seamless failover to Web Bluetooth for local management.
- **Text-to-Speech (TTS) Voice Announcements:** Interfacing an I2S digital audio amplifier (e.g., MAX98357A) and speaker to vocalize urgent emergency notices automatically.

References

The following references follow the prescribed IEEE format:

[1] Bluetooth SIG, "Bluetooth Core Specification Version 5.2," Bluetooth Special Interest Group, Kirkland, WA, Tech. Rep., 2020.

[2] Espressif Systems, "ESP32 Technical Reference Manual," Version 4.8, Shanghai, China: Espressif Systems Co., Ltd., 2023.

[3] S. Kumar and R. Singh, "IoT-Based Intelligent Digital Notice Board Using Bluetooth Low Energy and Mobile Application," in Proc. IEEE Int. Conf. Advances in Computing, Communications and Informatics (ICACCI), 2019, pp. 412–417.

[4] W3C, "Web Bluetooth Community Group Report," World Wide Web Consortium, Draft Community Group Report, 2023. [Online]. Available: https://webbluetoothcg.github.io/web-bluetooth/

[5] Solomon Systech, "SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller," Solomon Systech Ltd., Datasheet Rev 1.5, 2018.

[6] J. Bank, Node.js and Express.js in Production Architecture, Sebastopol, CA: O'Reilly Media, 2022.

[7] MongoDB Inc., "MongoDB Architecture and Indexing Strategies Guide," New York, NY: MongoDB Whitepaper, 2023.

[8] M. A. Aravinda and M. S. Kumar, "Energy-Efficient Telemetry and Display Driving Using ESP32 Dual-Core Microcontroller," IEEE Trans. Consumer Electron., vol. 68, no. 3, pp. 245–253, Aug. 2022.

[9] D. Harrison, "Responsive Web Design and Device Communications via Modern Browser APIs," IEEE Pervasive Comput., vol. 21, no. 2, pp. 58–66, 2022.

[10] A. N. Prasad, K. A. Mamun, and F. R. Islam, "Smart Campus Communication Systems Using Low-Power Wireless Networks," in Proc. IEEE APWC on CSE, 2020, pp. 18–25.

[11] Adafruit Industries, "Adafruit SSD1306 OLED Display Library Guide," Adafruit Learning System, New York, NY, Tech. Rep., 2022.

[12] FreeRTOS, "FreeRTOS Real-time Kernel Reference Manual," Real Time Engineers Ltd., Tech. Rep., 2023.

Appendix

Appendix A: Complete ESP32 Edge Firmware Source Code

The working Arduino C++ sketch (ESP32_BLE_Notice_Board.ino) compiles cleanly with the official ESP32 Arduino Core v3, managing BLE GATT callbacks, NVS persistence, priority sorting, SSD1306 graphics, and LEDC buzzer PWM:

```
// --- ESP32_BLE_Notice_Board.ino (Summary Snippet) ---
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Preferences.h>
#include <ArduinoJson.h>

#define SERVICE_UUID        "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
#define CHARACTERISTIC_UUID "beb5483e-36e1-4688-b7f5-ea07361b26a8"
#define PIN_GREEN_LED 25
#define PIN_RED_LED   26
#define PIN_BUZZER    27

Adafruit_SSD1306 display(128, 64, &Wire, -1);
Preferences preferences;

void setup() {
  Serial.begin(115200);
  pinMode(PIN_GREEN_LED, OUTPUT);
  pinMode(PIN_RED_LED, OUTPUT);
  ledcAttach(PIN_BUZZER, 2000, 10);
  Wire.begin(21, 22);
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  loadNoticesFromNVS();
  initBLE();
}

void loop() {
  handleBuzzerStateMachine();
  renderOLEDDisplay();
  delay(20);
}
```

Appendix B: REST API & BLE JSON Communication Payloads

1. Notice Creation POST Request Payload (/api/notices):

```
{
  "title": "End-Semester Lab Examination",
  "content": "IoT Practical exam scheduled in Lab 4.",
  "priority": "urgent",
  "scheduledFor": "2026-11-03T09:00:00.000Z",
  "expiresAt": "2026-11-05T17:00:00.000Z"
}
```

2. BLE Full Stack Synchronization Command (GATT Write Payload):

```
{
  "cmd": "sync",
  "notices": [
    { "title": "Fire Alarm Drill at 3 PM", "priority": "urgent" },
    { "title": "Guest Lecture in Hall B", "priority": "high" },
    { "title": "Library Open Until Midnight", "priority": "normal" },
    { "title": "Cafeteria Menu Updated", "priority": "low" }
  ]
}
```

3. BLE Single Notice Deletion Command:

```
{
  "cmd": "delete",
  "title": "Fire Alarm Drill at 3 PM"
}
```

Appendix C: Hardware Pin Mapping Matrix

Table C.1 provides the complete pin interconnect matrix for the ESP32 edge node.

ESP32 Pin	Peripheral Mode	Connected Component	Signal Type & Circuit Notes
GPIO 21	I2C SDA	SSD1306 0.96-inch OLED	3.3V I2C Serial Data line at standard address 0x3C.
GPIO 22	I2C SCL	SSD1306 0.96-inch OLED	3.3V I2C Serial Clock line (400 kHz Fast-Mode).
GPIO 27	LEDC PWM Channel	Active Piezo Buzzer	2000–2700 Hz acoustic tone; 50% duty cycle on urgent alert.

GPIO 25	Digital Output	Green Status LED	3.3V active HIGH through 220Ω series current-limiting resistor.
GPIO 26	Digital Output	Red Alert LED	3.3V active HIGH through 220Ω series current-limiting resistor.
3V3 Pin	Regulated Power	OLED Display VCC	Regulated 3.3V DC power rail from ESP32 onboard LDO.
GND Pin	Ground Plane	OLED, LEDs, Buzzer GND	Common electrical system ground reference plane.